Universidad Galileo	Guatemala 28 de octubre 2026
Facultad: FISICC	Alumno: Luis Cordova
Curso: Microprocesadores	Carnet: 23012122
Sección: A	Hora de Laboratorio: 1:00 a 1:50
Auxiliar: Brandon Eduardo Mendoza Espinoza	Día de Laboratorio: Lunes

Laboratorio No. 9 - PWM

Resumen:

Esta práctica junta dos módulos del STM32G431 que hasta ahora se habían trabajado por separado: el ADC y uno de los temporizadores. El convertidor lee dos entradas, un potenciómetro y un fotoresistor, y el temporizador produce una señal PWM que controla el brillo de un LED [4].

El resultado final es una luz automática con regulador: el potenciómetro fija el nivel de brillo y lo refleja en cuatro LEDs que se encienden por tramos, mientras que el fotoresistor decide si el LED regulado enciende o no. Con luz ambiente el LED queda apagado; al oscurecerse enciende con el brillo que marque el potenciómetro.

A diferencia de la práctica anterior, el ADC ya no se consulta en un lazo de espera sino que avisa por interrupción cada vez que termina una conversión, y todo el programa está escrito en C.

Objetivos:

- Configurar el ADC del STM32G431 para que trabaje con interrupciones, leyendo dos canales en secuencia continua.
- Generar una señal PWM con uno de los temporizadores del microcontrolador y controlar con ella el brillo de un LED.
- Combinar ambas lecturas para que el LED regulado encienda únicamente en la oscuridad y con el nivel de intensidad configurado con el potenciómetro.

Teoría:

Un pin digital solo entrega dos voltajes, 0 V o 3.3 V, así que no hay estado intermedio que produzca medio brillo. La modulación por ancho de pulso resuelve esa limitación conmutando el pin a una frecuencia lo bastante alta como para que el ojo no distinga los pulsos y perciba solo el promedio. La fracción de cada ciclo que el pin permanece en alto es el ciclo de trabajo, y es lo que el observador interpreta como brillo.

Los temporizadores del STM32G431 generan esa señal por hardware, sin que el procesador intervenga. El bloque tiene un contador que avanza con cada pulso de su reloj y se reinicia al llegar al valor del registro de autorrecarga, TIMx_ARR. Ese reinicio marca el flanco positivo y fija el período de la señal. En paralelo, el contador se compara de forma permanente con el registro de comparación del canal, TIMx_CCRn: mientras el contador sea menor, la salida se mantiene en alto; al igualarlo o superarlo, se genera el flanco negativo [4].

El enunciado enumera los temporizadores de toda la familia STM32G4: tres de control avanzado de 16 bits, TIM1, TIM8 y TIM20; dos de propósito general de 32 bits, TIM2 y TIM5; cinco de propósito general de 16 bits, TIM3, TIM4, TIM15, TIM16 y TIM17; y dos básicos, TIM6 y TIM7 [4]. El STM32G431 no los trae todos: según el archivo de definiciones del dispositivo, en este microcontrolador están disponibles TIM1 y TIM8 de control avanzado, TIM2 de 32 bits, TIM3, TIM4, TIM15, TIM16 y TIM17 de propósito general, y los dos básicos, pero no TIM5 ni TIM20 [8].

Los de control avanzado incorporan un circuito de protección pensado para el manejo de potencia que mantiene todas sus salidas inhibidas hasta que se habilita el bit MOE del registro TIMx_BDTR [1].

Del lado del ADC, la diferencia respecto a la práctica anterior está en cómo se entera el programa de que hay un dato listo. El sondeo consiste en preguntar repetidamente por una bandera, lo que mantiene al procesador ocupado en un lazo de espera. Con interrupciones el convertidor solicita atención por su cuenta: el procesador abandona lo que esté haciendo, atiende la rutina de servicio y regresa al punto en que estaba [1].

El ADC distingue dos eventos. La bandera EOC se levanta al terminar cada conversión individual, mientras que EOS lo hace al completarse la secuencia entera configurada en ADCn_SQR1. Esa distinción es la que permite leer varios canales por vuelta sabiendo cuál dato corresponde a cuál entrada [1].

El fotoresistor, por último, es una resistencia cuyo valor disminuye al recibir luz. Como el ADC mide voltaje y no resistencia, se lo conecta formando un divisor con una resistencia fija; el nodo intermedio entrega entonces un voltaje que varía con la iluminación.

Resultados prácticos:

A continuación se presentan las capturas obtenidas durante el desarrollo y la ejecución de la práctica.

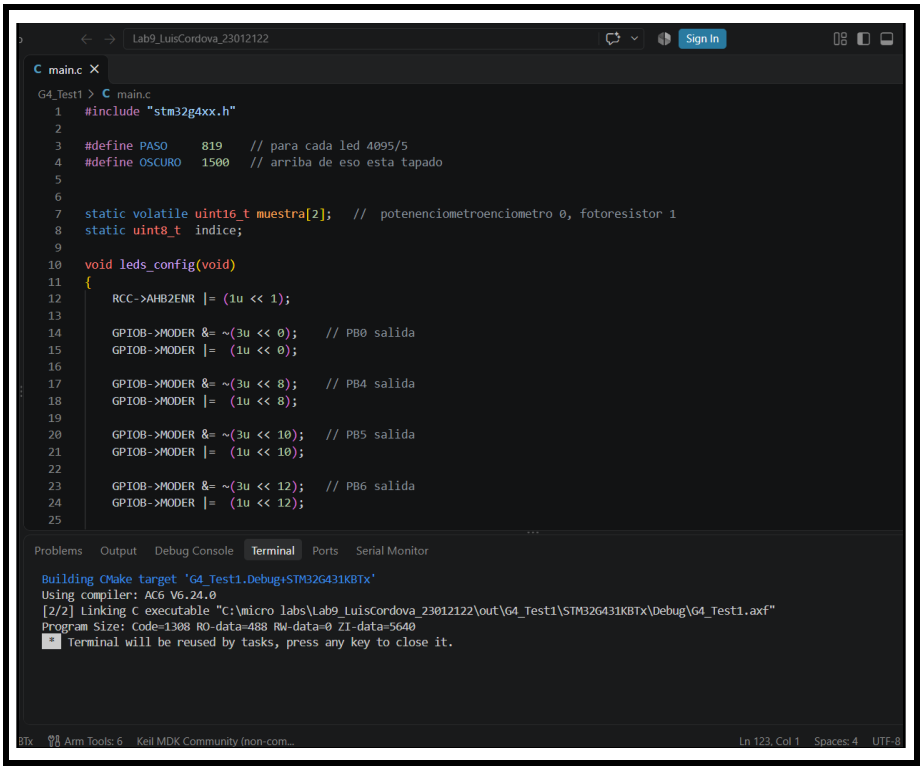


Ilustración 1- constantes del programa y subrutina leds_config, con los cuatro pines del puerto B configurados como salida.

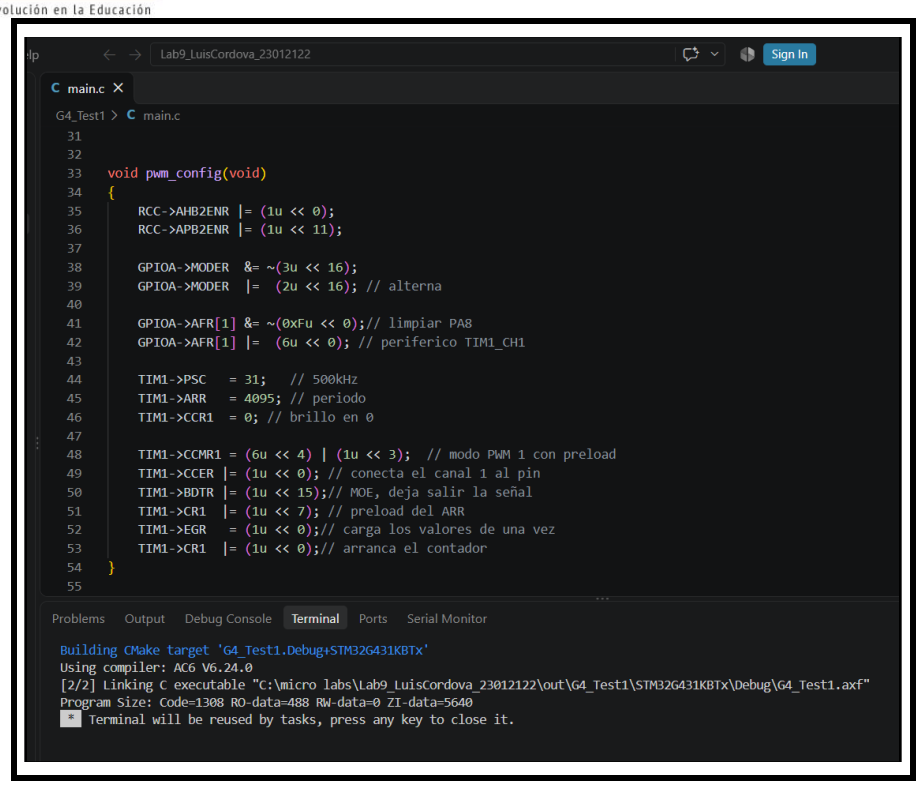
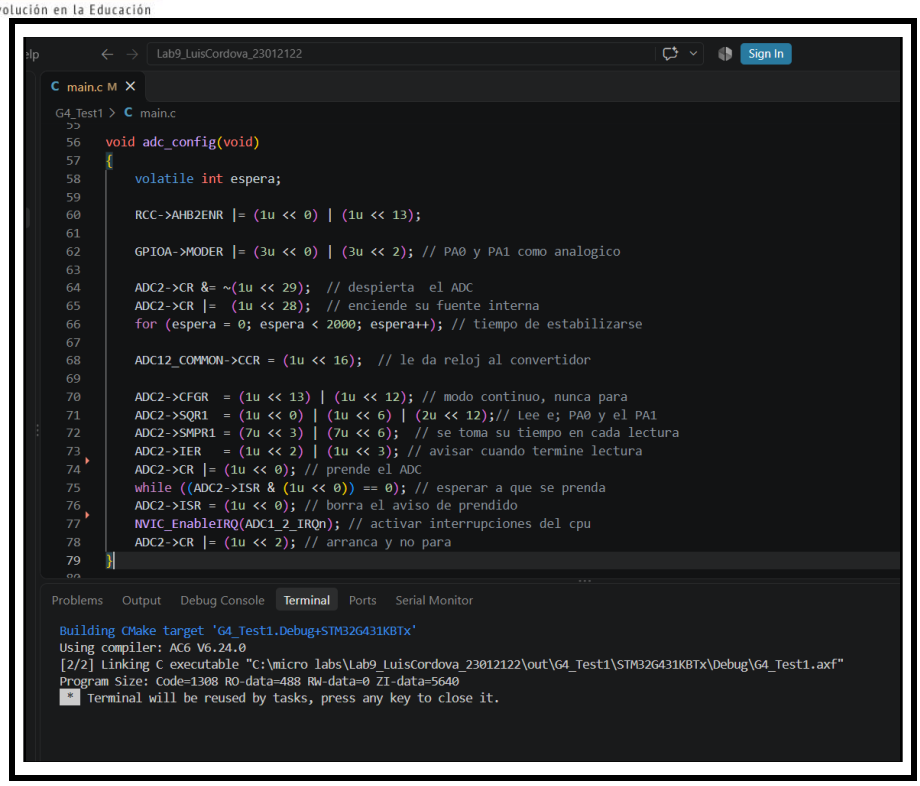


Ilustración 2- subrutina pwm_config: PA8 en función alterna, configuración de TIM1 y arranque del contador.




```
56  void adc_config(void)
57  {
58      volatile int espera;
59
60      RCC->AHB2ENR |= (1u << 0) | (1u << 13);
61
62      GPIOA->MODER |= (3u << 0) | (3u << 2); // PA0 y PA1 como analogico
63
64      ADC2->CR &= ~(1u << 29);  // despierta  el ADC
65      ADC2->CR |=  (1u << 28);  // enciende su fuente interna
66      for (espera = 0; espera < 2000; espera++); // tiempo de estabilizarse
67
68      ADC12_COMMON->CCR = (1u << 16);  // le da reloj al convertidor
69
70      ADC2->CFGR  = (1u << 13) | (1u << 12); // modo continuo, nunca para
71      ADC2->SQR1  = (1u << 0) | (1u << 6) | (2u << 12);// Lee e; PA0 y el PA1
72      ADC2->SMPR1 = (7u << 3) | (7u << 6);  // se toma su tiempo en cada lectura
73      ADC2->IER   = (1u << 2) | (1u << 3); // avisar cuando termine lectura
74      ADC2->CR |= (1u << 0); // prende el ADC
75      while ((ADC2->ISR & (1u << 0)) == 0); // esperar a que se prenda
76      ADC2->ISR = (1u << 0); // borra el aviso de prendido
77      NVIC_EnableIRQ(ADC1_2_IRQn); // activar interrupciones del cpu
78      ADC2->CR |= (1u << 2); // arranca y no para
79  }
```

Ilustración 3- subrutina adc_config: relojes, pines analógicos, salida del deep power down y secuencia de dos canales.

```
81  void leds_write(uint16_t potenenciometro)
82  {
83      GPIOB->ODR &= ~(1u << 0);
84      GPIOB->ODR &= ~(1u << 4);
85      GPIOB->ODR &= ~(1u << 5);
86      GPIOB->ODR &= ~(1u << 6);
87
88      if (potenenciometro >= PASO)       GPIOB->ODR |= (1u << 0);
89      if (potenenciometro >= 2 * PASO)   GPIOB->ODR |= (1u << 4);
90      if (potenenciometro >= 3 * PASO)   GPIOB->ODR |= (1u << 5);
91      if (potenenciometro >= 4 * PASO)   GPIOB->ODR |= (1u << 6);
92  }
93
94
```

Ilustración 4- subrutina leds_write: apagado previo y encendido acumulativo según los umbrales.

```
94
95  void ADC1_2_IRQHandler(void)
96  {
97      if (ADC2->ISR & (1u << 2)) { // termino 1 lectura
98          muestra[indice] = (uint16_t)ADC2->DR;
99          indice ^= 1u; // cambia el indice
100     }
101
102     if (ADC2->ISR & (1u << 3)) { // termino las 2 lecturas
103         ADC2->ISR = (1u << 3);
104         indice = 0; // reinicia el indice
105     }
106 }
```

Ilustración 5- rutina de servicio de la interrupción del ADC, con el manejo de las banderas EOC y EOS.

```
Lab9_LuisCordova_23012122                                   Sign In
C main.c M
G4_Test1 > C main.c
106 }
107
108 int main(void)
109 {
110
111     leds_config();
112     pwm_config();
113     adc_config();
114
115     while (1) {
116
117         uint16_t potenenciometro = muestra[0];
118         uint16_t luz = muestra[1];
119
120         leds_write(potenenciometro);
121
122         if (luz > OSCURO)
123             TIM1->CCR1 = potenenciometro; // no luz = prender
124         else
125             TIM1->CCR1 = 0; // Luz = apagar
126     }
127 }

Problems  Output  Debug Console  Terminal  Ports  Serial Monitor

Building CMake target 'G4_Test1.Debug+STM32G431KBTx'
Using compiler: AC6 V6.24.0
[2/2] Linking C executable "C:\micro labs\Lab9_LuisCordova_23012122\out\G4_Test1\STM32G431KBTx\Debug\G4_Test1.axf"
Program Size: Code=1308 RO-data=488 RW-data=0 ZI-data=5640
*  Terminal will be reused by tasks, press any key to close it.
```

Ilustración 6- función main con las tres configuraciones y el lazo principal.

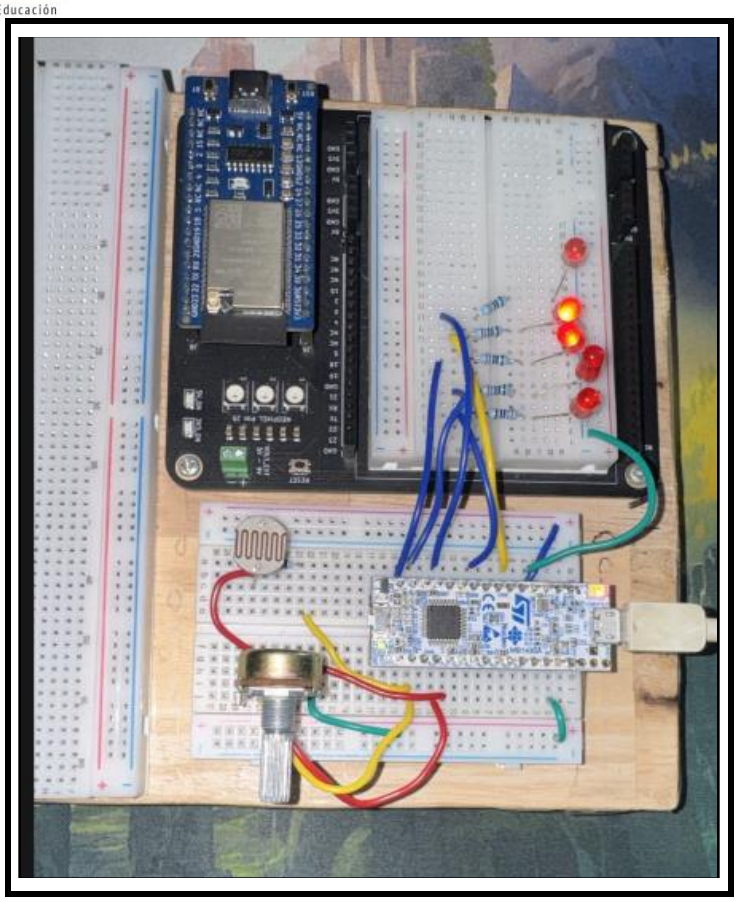


Ilustración 7- circuito armado en protoboard: la tarjeta Nucleo-G431KB, el potenciómetro, el fotoresistor con su divisor y los LEDs.

Video de la práctica:

En el siguiente enlace se muestra el funcionamiento de la aplicación y la modificación del paso 4.

https://youtube.com/shorts/FcA5U1285yA

Resultados Teóricos:

Diseño del circuito

El esquemático del enunciado corresponde a la tarjeta de evaluación del STM32F303 y ubica los LEDs en el puerto E, el potenciómetro en PC1 y el fotoresistor en PA0. La Nucleo-G431KB solo expone pines de los puertos A y B, más PF0 y PF1, así que la asignación tuvo que rehacerse por completo [2]. El armado se hizo en protoboard.

Los cuatro LEDs indicadores conservaron los mismos pines de la práctica anterior, con lo que ese cableado no hubo que tocarlo. El potenciómetro se movió a PA0 para dejar PA1 libre para el fotoresistor, pues ambos deben pertenecer al mismo convertidor para poder leerse en una sola secuencia. La asignación final quedó así:

Elemento	Pin	Rótulo	Función
Potenciómetro	PA0	A0	ADC2_IN1
Fotoresistor	PA1	A1	ADC2_IN2
LD1	PB0	D3	salida digital
LD2	PB4	D12	salida digital
LD3	PB5	D11	salida digital
LD4	PB6	D6	salida digital
LED regulado	PA8	D9	TIM1_CH1

Cada LED va del pin a una resistencia de 220 Ω, luego al LED y a tierra, de manera que enciende con un uno lógico. Con un LED rojo, de unos 2 V de caída, la corriente resulta:

$$I = (3.3\ V - 2.0\ V) / 220\ \Omega \approx 5.9\ mA$$

El fotoresistor no puede conectarse solo, porque entrega resistencia y no voltaje. Se armó un divisor con una resistencia fija de 9.1 kΩ entre los 3.3 V y el nodo de medición, y el fotoresistor entre ese nodo y tierra. Medido con el multímetro, el componente presentó alrededor de 3 kΩ con iluminación y 10 kΩ al taparlo, valores que justifican el orden de magnitud de la resistencia escogida.

Elección del temporizador y del pin

El pin de salida del PWM no puede escogerse libremente: tiene que ser uno que la tabla de funciones alternas conecte con un canal de salida de algún temporizador. En la Nucleo-G431KB, PA8 está asociado a TIM1_CH1 a través de la función alterna 6 [8].

TIM1 es además un temporizador de control avanzado, que es la familia a la que apunta el enunciado al mencionar el registro TIMx_BDTR entre los que hay que configurar [4]. Esa mención no es casual: en los temporizadores avanzados toda la configuración puede estar correcta y la señal no aparecer en el pin si no se habilita el bit MOE, que libera el circuito de protección de salidas [1].

La conexión del pin al temporizador requiere dos pasos independientes. Primero se coloca PA8 en modo de función alterna escribiendo 10 en sus dos bits de GPIOA_MODER, con lo que el pin deja de depender del registro de salida. Después se indica cuál de las dieciséis funciones alternas posibles le corresponde, escribiendo el valor 6 en los cuatro bits de PA8 dentro de GPIOA_AFRH [1].

Frecuencia de la señal

El período de la señal lo determinan en conjunto el preescalador y el registro de autorrecarga. La relación que los vincula con la frecuencia deseada es:

$$ARR = fclk / ((PSC + 1) \times fPWM) - 1$$

Despejada en el sentido inverso, que es el que interesa para documentar la frecuencia obtenida a partir de los valores programados:

$$fPWM = fclk / ((PSC + 1) \times (ARR + 1))$$

El valor de ARR se fijó primero, en 4095, por una razón ajena a la frecuencia y que se explica en el apartado siguiente. Con el reloj del sistema en 16 MHz y el preescalador en 31, la sustitución da:

$$fPWM = 16\,000\,000 / (32 \times 4096) = 122.07 \text{ Hz}$$

Una primera versión del programa trabajó con un preescalador de 3, lo que arrojaba 976.56 Hz. Durante la revisión se señaló que esa frecuencia es excesiva para la aplicación, tomando como referencia que las lámparas conectadas a la red parpadean sin que nadie lo note. El valor se bajó en consecuencia.

Vale la pena precisar el número, porque el que importa no es el de la red sino el doble. Una lámpara incandescente alimentada con corriente alterna de 60 Hz se atenúa en los dos semiciclos, de modo que su parpadeo real ocurre 120 veces por segundo. Los 122.07 Hz programados quedan justamente en ese orden, con lo que la señal es imperceptible sin conmutar de más.

La distinción tiene además una consecuencia práctica. El filamento de una lámpara se queda caliente entre pulso y pulso, mientras que un LED se apaga de forma instantánea. Por ese motivo un LED exige algo más de frecuencia que una lámpara para verse igual de estable, sobre todo si se lo mueve, y bajar hasta los 60 Hz de la red habría sido arriesgado.

Escala del convertidor y correspondencia con el brillo

El ADC se dejó en su resolución máxima de 12 bits, de modo que cada conversión entrega un número entre 0 y 4095 [1]. El registro de comparación del temporizador acepta cualquier valor hasta el de autorrecarga.

Fijar ARR en 4095 hace que ambos rangos coincidan exactamente, y el valor leído del potenciómetro puede escribirse en TIM1_CCR1 sin ninguna conversión intermedia. Con un ARR distinto habría que escalar el dato en cada vuelta, con el costo y el redondeo que eso implica. La correspondencia queda:

Potenciómetro	Voltaje en PA0	CCR1	Ciclo de trabajo
0	0.00 V	0	0 %
1024	0.83 V	1024	25 %
2048	1.65 V	2048	50 %
4095	3.30 V	4095	100 %

Lectura de dos canales en una sola secuencia

El potenciómetro y el fotoresistor se leen con el mismo convertidor, ADC2, aprovechando que admite una secuencia de conversiones sobre canales distintos. Esa secuencia se declara en ADCn_SQR1, registro que concentra tres datos en una sola escritura [1]:

Campo	Valor	Significado
L	1	dos conversiones por secuencia
SQ1	1	la primera es el canal 1, PA0
SQ2	2	la segunda es el canal 2, PA1

El campo de longitud se escribe siempre con una unidad menos que la cantidad real de conversiones. Junto con el bit CONT de ADCn_CFGR, que reinicia la secuencia en cuanto termina, el convertidor queda alternando indefinidamente entre ambas entradas sin que el programa vuelva a intervenir.

Al tiempo de muestreo de los dos canales se le asignó el valor más largo disponible en ADCn_SMPR1. La entrada del convertidor incorpora un capacitor que debe cargarse al voltaje del pin antes de iniciar la medición, y con fuentes de resistencia alta como un potenciómetro de 10 kΩ o el divisor del fotoresistor ese proceso requiere más tiempo del que ofrece el valor por omisión [1].

Las dos banderas de la interrupción

La rutina de servicio se llama ADC1_2_IRQHandler, nombre que no es arbitrario: coincide con la entrada correspondiente de la tabla de vectores declarada en el archivo de arranque del dispositivo. Cualquier otro nombre compila sin error pero deja la interrupción sin atender.

Dentro de la rutina se consultan las dos banderas por separado, y cada una cumple una función distinta:

- EOC indica que terminó una conversión. El dato se recoge de ADCn_DR y se guarda en el arreglo de muestras, alternando el índice para que la siguiente caiga en la otra posición. Leer el registro de datos limpia la bandera por hardware, de modo que no hace falta borrarla.
- EOS indica que terminó la secuencia completa. Esta bandera sí debe borrarse escribiendo un uno en su posición, y se aprovecha el momento para devolver el índice a cero.

Ese reinicio del índice al cerrar cada secuencia funciona como seguro. Aunque por algún motivo se perdiera una conversión y el índice quedara desfasado, la siguiente marca de fin de secuencia lo vuelve a alinear, de forma que la posición cero siempre corresponde al potenciómetro y la uno al fotoresistor.

El arreglo de muestras se declaró volatile. La rutina de interrupción lo escribe y el lazo principal lo lee, y sin ese calificativo el compilador podría suponer que su contenido no cambia entre dos lecturas consecutivas y conservar el valor en un registro en lugar de volver a consultarlo.

Umbral de oscuridad

Con el divisor armado se midió el voltaje del nodo en las dos condiciones extremas de uso. Convertidos a cuentas del ADC con la referencia de 3.3 V, los valores fueron:

Condición	Voltaje medido	Cuentas
Iluminación de la habitación	2.7 V	3350
Fotoresistor cubierto	3.2 V	3971

El sentido de la variación es el esperado: al recibir luz el fotoresistor baja su resistencia, el nodo se acerca a tierra y la lectura disminuye. La comparación del programa se escribió en consecuencia, tomando los valores altos como señal de oscuridad.

El umbral quedó fijado en 1500 cuentas, equivalentes a 1.21 V, que corresponde a iluminación franca sobre el sensor. Es un único valor en una constante, de modo que ajustar la sensibilidad no exige tocar el circuito.

Indicación del nivel con los cuatro LEDs

El recorrido del potenciómetro se reparte en quintos, igual que en la práctica anterior pero sobre la escala completa de 12 bits:

$4095 / 5 = 819 \rightarrow 819, 1638, 2457$ y 3276

La subrutina apaga primero los cuatro LEDs y después recorre los cuatro umbrales encendiendo uno más en cada uno. El apagado previo no se puede quitar: las comparaciones únicamente encienden, nunca apagan, de modo que sin él un LED ya encendido permanecería así aunque el potenciómetro bajara. El resultado es una barra acumulativa:

Recorrido	Valor del ADC	LEDs encendidos
0 a 1/5	0 a 818	ninguno
1/5 a 2/5	819 a 1637	LD1
2/5 a 3/5	1638 a 2456	LD1 y LD2
3/5 a 4/5	2457 a 3275	LD1, LD2 y LD3
4/5 a 1	3276 a 4095	los cuatro

Modificación del paso 4

El procedimiento pide borrar las líneas del ciclo infinito y volver a cargar el programa. Con la estructura empleada, el ciclo contiene la lectura del arreglo de muestras, la actualización de los LEDs indicadores y la decisión sobre el LED regulado.

Lo que se observa al quitarlo es interesante, porque no todo se detiene. El temporizador continúa generando la señal con el último valor que quedó en TIM1_CCR1, y el convertidor sigue trabajando y almacenando ambas muestras por interrupción. Lo único que deja de ocurrir es la actualización: los cuatro LEDs indicadores quedan congelados en el patrón que tenían y el LED regulado conserva su brillo, sin responder ya al potenciómetro ni al fotoresistor.

Esa diferencia muestra con claridad qué parte del sistema depende del procesador y cuál no. La generación de la señal y la conversión están delegadas al hardware y siguen andando aunque el ciclo quede vacío; la lógica que vincula a ambas es la que vive en el programa.

Conclusiones:

- Delegar trabajo al hardware cambia la forma del programa. El temporizador produce la señal de PWM y el convertidor realiza sus conversiones sin intervención del procesador, de modo que el programa se reduce a escribir un número en un registro de comparación y a decidir cuándo hacerlo.
- Las interrupciones separan la obtención del dato del momento en que se usa. El convertidor avisa cuando tiene un resultado en lugar de obligar a esperarlo en un lazo, y la distinción entre las banderas de fin de conversión y de fin de secuencia es la que permite leer dos entradas por vuelta sin confundir cuál corresponde a cuál.
- La frecuencia de una señal de PWM no se escoge lo más alta posible sino lo suficientemente alta. Subirla más allá de lo que el ojo distingue solo agrega conmutaciones sin mejorar nada, y el criterio útil resulta ser el parpadeo de una lámpara de la red, que ocurre 120 veces por segundo.

Bibliografía:

[1] STMicroelectronics, «STM32G4 series advanced Arm-based 32-bit MCUs», Reference manual RM0440, Rev. 9, marzo de 2025, capítulos 9, 21 y 27. [En línea]. Disponible en: https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

[2] STMicroelectronics, «STM32G4 Nucleo-32 board (MB1430)», User manual UM2397, Rev. 2, septiembre de 2019. [En línea]. Disponible en: https://www.st.com/resource/en/user_manual/um2397-stm32g4-nucleo32-board-mb1430-stmicroelectronics.pdf

[3] STMicroelectronics, «STM32 Cortex-M4 MCUs and MPUs programming manual», Programming manual PM0214. [En línea]. Disponible en: https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf

[4] J. Fajardo, «Práctica de Laboratorio #9: Pulse Width Modulation», Microprocesadores, Universidad Galileo, octubre de 2026.

[5] STMicroelectronics, «STM32G431x6/x8/xB datasheet», DS12589, tabla de definiciones de pines. [En línea]. Disponible en: https://www.st.com/resource/en/datasheet/stm32g431kb.pdf

[6] STMicroelectronics, «stm32g4xx_hal_driver», funciones HAL_ADC_Init y HAL_TIM_PWM_Init. [En línea]. Disponible en: https://github.com/STMicroelectronics/stm32g4xx_hal_driver

[7] Arm Limited, «Arm Compiler for Embedded User Guide», Arm Compiler 6. [En línea]. Disponible en: https://developer.arm.com/documentation/

[8] STMicroelectronics, «cmsis_device_g4», archivo stm32g431xx.h, definiciones de periféricos y funciones alternas del dispositivo. [En línea]. Disponible en: https://github.com/STMicroelectronics/cmsis_device_g4